

Project 1 — Complete Explanation

Multi-Agent Customer Support System

Big Picture

When a customer sends a message (via WhatsApp, email, SMS, or any channel), your system:

- Classifies what they want
- Routes to the right specialist agent
- Answers or acts (booking, refund, FAQ)
- Validates the answer for accuracy
- Escalates to a human when AI should not decide alone
- Logs everything for audit

This is exactly what NEXIO24 asked for in the job description.

The 7 Workflows Explained

1. Triage Agent

What it does: Reads a message, decides what the customer wants.

Example:

Input: "I want to book a table for 4"

Output: {"intent": "booking", "confidence": 0.9}

Why: The routing decision is independent of actually doing the task. Cheap, fast, can use a small model.

Job requirement covered: Multi-agent orchestration, LLM classification.

2. RAG Ingestion

What it does: Loads your FAQ document into a vector database.

Steps:

- Reads faq.txt from disk
- Splits it into 500-character chunks

- Converts each chunk to an embedding (a 1536-number vector)
- Stores chunk + vector in Supabase documents table

Why: So the Support Agent can search by meaning, not just keywords.

Job requirement covered: RAG, embeddings, vector search.

3. Support Agent

What it does: Answers customer questions using the FAQ.

Steps:

- Receives question via webhook
- Embeds the question
- Vector searches documents for top 3 matches
- Builds context from those chunks
- LLM generates an answer with confidence score
- If confidence < 0.7 → escalate to human
- Log to conversations table

Example:

Input: "What are your opening hours?"

Output: {"status": "answered", "answer": "We are open 24/7...", "confidence": 1}

Why: Read-only agent. Cannot change bookings or users. This is permission isolation.

Job requirement covered: RAG, prompt engineering, human-fallback logic, tool calling.

4. Escalation Agent

What it does: Handles requests that AI should not decide (refunds, emergencies).

Steps:

- Receives escalation via webhook
- Logs the escalation reason
- Builds a ticket with priority (normal or high for emergencies)
- Notifies human (placeholder for email/Slack)
- Returns ticket ID

Example:

```
{"message": "I want a refund", "reason": "high_value_refund"}  
{"ticket_id": "ESC-...", "status": "pending_human", "priority": "normal"}
```

Why: Centralized human handoff. Single point for audit and SLA tracking.

Job requirement covered: Human-fallback, escalation paths, ticket management.

5. Booking Agent

What it does: Extracts booking details from natural language and creates reservations.

Steps:

- Receives message via webhook
- LLM extracts date, time, party_size as JSON
- Checks if any field is missing
- If missing → asks user for the missing fields
- If complete → inserts into bookings table
- Logs to conversations

Example:

"Book a table for 4 tomorrow at 7pm"

```
{"status": "confirmed", "date": "2026-09-30", "time": "19:00", "party_size": 4}
```

Why: Isolated write permissions. Can only write to bookings. Cannot read other users' data.

Job requirement covered: Tool/function calling, structured extraction, conditional logic.

6. QA Agent

What it does: Validates other agents' outputs. Rejects hallucinations.

Two-Layer Validation: Two-Layer Validation: (1) Keyword Check (deterministic): Rejects if <50% of response keywords appear in context. No LLM involved. (2) LLM QA (strict prompt): Rejects if any claim is not in the context. Both must approve. The LLM cannot override the deterministic check.

Example:

```
response = "We offer free helicopter rides", context = "We are open 24/7"  
{"status": "rejected", "reason": "keyword_coverage 0%"}
```

Why: Centralized quality gate. Hybrid deterministic + LLM is stronger than LLM alone.

Job requirement covered: Guardrails, monitoring, QA, hallucination prevention.

7. Supervisor

What it does: Single entry point. Routes messages to the right agent.

Steps:

- Receives message via webhook
- Fast Triage (regex) classifies intent:
 - • Contains book|reserve|table → booking
 - • Contains refund|complaint|angry → complaint
 - • Contains fire|emergency → emergency
 - • Otherwise → faq
- Routes to: Booking Agent (intent = booking), Support Agent (intent = faq), Escalation Agent (intent = complaint or emergency)
- Returns the agent's response

Example:

"Book a table for 4 tomorrow at 7pm"

```
{"status": "confirmed", ...}
```

Why: Single entry point for all channels. Agents stay decoupled. Routing logic is fast and free (regex, not LLM).

Job requirement covered: Multi-agent orchestration, routing, escalation paths.

Database Tables

conversations

Stores every message and agent action for audit.

Column	Purpose
request_id	Unique ID per request
user_id	Who sent it
channel	whatsapp / email / sms
message	Original text
intent	What they wanted
agent	Which agent handled it
response	What we replied
confidence	LLM confidence
created_at	Timestamp

documents

Stores FAQ chunks + embeddings for RAG.

Column	Purpose
content	500-char text chunk
embedding	1536-number vector
source	faq.txt
chunk_index	Order

bookings

Stores reservations.

Column	Purpose
user_id	Who booked
date / time	When
party_size	How many
status	confirmed

Key Design Decisions (Interview Answers)

Why 7 separate workflows?

Say *this*: "Permission isolation. Booking Agent can only write to bookings. Support Agent is read-only. If one agent is compromised, it cannot affect others. Also, agents can be scaled and updated independently."

Why two-layer QA?

Say *this*: "LLMs are too lenient. I tested and the LLM approved hallucinations. So I added a deterministic keyword check first. LLM cannot override it. This is the production pattern: deterministic guardrails + LLM reasoning."

Why regex for triage, not LLM?

Say *this*: "Triage is a routing decision. It needs to be fast and free. Regex handles 90% of cases. For the remaining 10%, the Support Agent's own confidence check escalates. This keeps cost low and latency under 1 second."

Why escalate to human?

Say *this*: "AI should not decide on refunds over policy, emergencies, or low-confidence cases. The system escalates with full context so a human can act immediately. This is human-in-the-loop design."

How do you control cost?

Say *this*: "In development I use OpenRouter's free models. In production, I would log token usage per request and set daily budget alerts. The conversations table already tracks this — Project 3 builds the dashboard."

What the System Handles

Scenario	Route	Result
Book a table	Booking Agent	Confirmed reservation
Ask about hours	Support Agent (RAG)	Cited answer
Refund request	Escalation Agent	Human ticket
Emergency	Escalation Agent	High-priority ticket
Hallucinated answer	QA Agent	Rejected

Low confidence	Support Agent	Escalated
Missing booking info	Booking Agent	Asks user